通过 USB 固化 Linux 系统至评估板

Revision History

Draft Date	Revision No.	Description
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 3
1 usb_serial_download 原理说明 5
2 通过 USB 固化 Linux 系统 5
2.1 固化 Linux 系统至 Micro SD 卡 6
2.2 固化 Linux 系统至 eMMC 7
2.3 从 Micro SD 或 eMMC 启动评估板 8
3 关键代码 8
3.1 burn_to_sd.bat、burn_to_emmc.bat 脚本 8
3.2 run_ramdisk.lst 脚本 10
3.3 sd.lst、emmc.lst 脚本 10
3.4 mkpartition.sh 脚本 12
更多帮助 14

前 言

本文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu 20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71

UUU(Universal Update Utility)系统固化工具包：1.5.141 版本

Linux SDK：Real-Time Edge Software 2.5

本文档主要介绍基于 Serial Downloader 模式下，使用 NXP 官方提供的 UUU(Universal Update Utility)系统固化工具包（版本为 1.5.141），通过评估板 USB3.0 DRD(USB1)接口固化 Linux 系统至 Micro SD 卡或板载 eMMC。

相关资料位于产品资料“4-软件资料\Demo\platform-demos\usb_serial_download\”目录下，具体说明如下。UUU 工具的 Github 仓库地址：https://github.com/NXPmicro/mfgtools/releases。

表 1

bin	uuu.exe	UUU 工具包可执行文件
doc	《UUU》	官方参考文档
scripts	brun_to_emmc.bat	Windows 脚本，用于调用 UUU 工具和 UUU 固化脚本
	brun_to_sd.bat	
	mkpartition.sh	分区脚本文件，用于对 eMMC 或 Linux 系统启动卡进行分区
	run_ramdisk.lst	预加载脚本，先加载内核、设备树、ramdisk 镜像，并在 RAM 启动 Linux 系统
	emmc.lst	UUU 固化脚本，用于固化 Linux 系统至 eMMC
	sd.lst	UUU 固化脚本，用于固化 Linux 系统至 Linux 系统启动卡
	Image	内核镜像
	tlimx8mp-evm.dtb	设备树镜像

src	LF_v5.15.71-2.2.0_images_IMX8MPEVK.zip	NXP 官方 BSP 镜像包

1 usb_serial_download 原理说明

usb_serial_download 示意框图如下图所示，其基本原理是先启动一次 Linux 系统，在系统上根据 eMMC 或 Micro SD 存储设备创建并格式化分区，将制卡工具包的系统部署至 eMMC 或 Micro SD 卡存储设备。

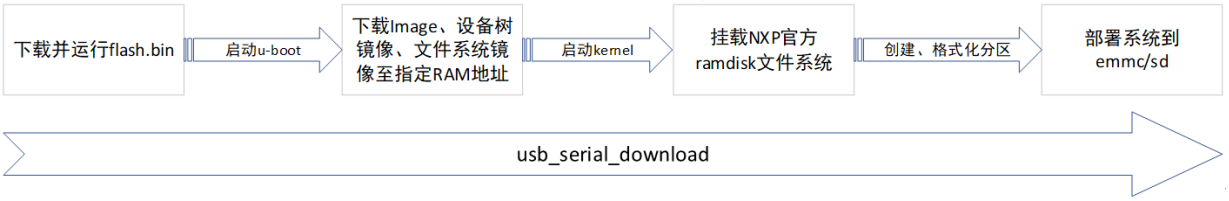

图 1

相关镜像说明如下表所示。

表 2

flash.bin	U-Boot 镜像，usb_serial_download 过程中会先解压系统制卡工具包，再从解压出来的制卡工具包中下载并启动 flash.bin，用于启动 U-Boot
Image、设备树镜像	在 scripts 目录下读取 RAM 指定的地址，用于启动内核
文件系统镜像	在 src 目录下提供 NXP 官方系统镜像压缩包，里面包含所需的文件系统镜像，Linux 内核启动后，挂载此文件系统，在系统上部署制卡工具包的系统至 eMMC 或 Micro SD 卡存储设备

2 通过 USB 固化 Linux 系统

请将 src 目录下的 NXP 官方 BSP 镜像包 LF_v5.15.71-2.2.0_images_IMX8MPEVK.zip 解压，将解压得到的 fsl-image-mfgtool-initramfs-imx_mfgtools.cpio.zst.u-boot 文件拷贝至 bin 目录。

图 2

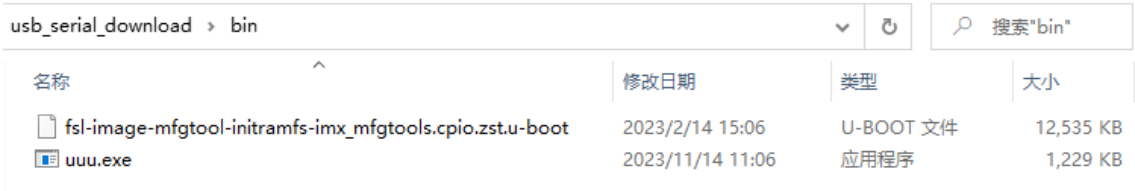

图 3

然后将产品资料“4-软件资料\Linux\Makesdboot\”目录下的 mksdboot_AA_BB_CC.tar.gz 制卡工具包拷贝至 bin 目录下。AA_BB_CC 为制卡工具包的版本号，请以实际为准。

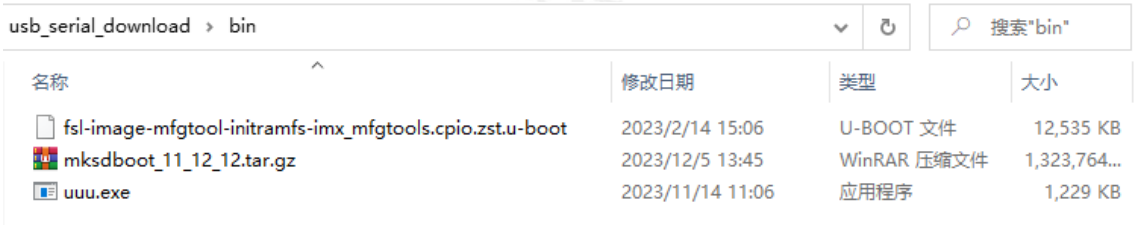

图 4

2.1 固化 Linux 系统至 Micro SD 卡

请将 Micro SD 卡插至评估板 Micro SD 卡槽，并通过 Type-C 线将评估板 USB3.0 DRD (USB1)接口与 PC 机连接，然后根据评估底板丝印将启动方式选择拨码开关拨为 1000(1~4)，此档位为 USB Serial Download 模式，再将评估板上电。

双击运行 scripts 目录下的 burn_to_sd.bat 脚本，Windows 会弹出 CMD 窗口，并开始固化 Linux 系统至 Micro SD 卡。约 35min 后完成固化，按任意键可退出 CMD 窗口。

备注：固化系统的过程中 USB3.0 DRD(USB1)接口会自动断开并重新连接，为避免 Ubuntu 虚拟机抢占 USB，测试时请关闭 Ubuntu 虚拟机。

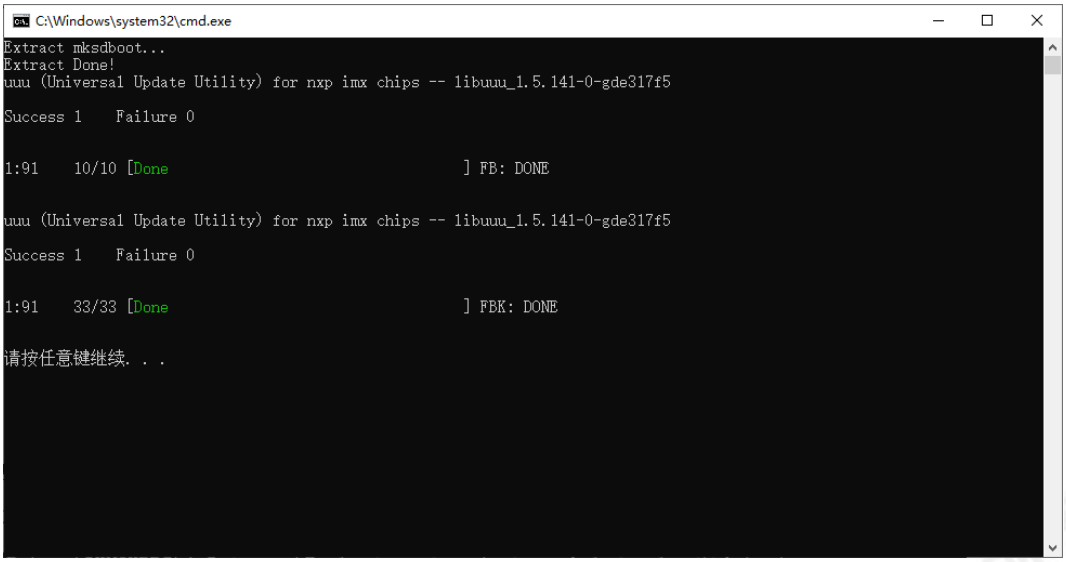

图 5

2.2 固化 Linux 系统至 eMMC

请将评估板 USB3.0 DRD(USB1)接口通过 Type-C 线连接至 PC 后，根据评估底板丝印将启动方式选择拨码开关拨为 1000(1~4)，此档位为 USB Serial Download 模式，再将评估板上电。

双击运行 scripts 目录下的 burn_to_emmc.bat 脚本，Windows 会弹出 CMD 窗口，并开始固化 Linux 系统至板载 eMMC。约 20min 后完成固化，按任意键可退出 CMD 窗口。

备注：固化系统的过程中 USB3.0 DRD(USB1)接口会自动断开并重新连接，为避免 Ubuntu 虚拟机抢占 USB，测试时请关闭 Ubuntu 虚拟机。

```
Extract mksdboot...
Extract Done!
uuu (Universal Update Utility) for nxp imx chips -- libuuu_1.5.141-0-gde317f5

Success 1    Failure 0

1:524   10/10 [Done                                ] FB: DONE

uuu (Universal Update Utility) for nxp imx chips -- libuuu_1.5.141-0-gde317f5

Success 1    Failure 0

1:524   25/25 [Done                                ] FBK: DONE

请按任意键继续. . .
```

图 6

2.3 从 Micro SD 或 eMMC 启动评估板

请根据评估底板丝印将启动方式选择拨码开关拨至 Micro SD 或 eMMC 启动模式，即可正常启动 Linux 系统。

```
NXP Real-time Edge Distro 2.5 IMX8MP-Tronlong ttymxc1
IMX8MP-Tronlong login: root (automatic login)
root@IMX8MP-Tronlong:~#
```

图 7

3 关键代码

3.1 burn_to_sd.bat、burn_to_emmc.bat 脚本

由于 burn_to_sd.bat、burn_to_emmc.bat 脚本文件作用类似，因此以 burn_to_emmc.bat 为例进行说明。

（1） 解压 mksdboot。

```
::Extract the mksdboot
echo Extract mksdboot...
move ..\bin\mksdboot*.tar.gz ..\bin\mksdboot.tar.gz >nul
tar -zxf ..\bin\mksdboot.tar.gz -C .
move .\mksdboot* .\mksdboot  >nul
echo Extract Done!
```

图 8

（2） 设置本地变量。

```
::Set local var
SET path=mksdboot
SET rootfs=%Path%\filesystem\rootfs\*.tar.gz
SET modules=%Path%\filesystem\modules\*.tar.gz
SET ubootfile=%Path%\boot\flash.bin
SET kernel=%Path%\boot\Image
SET devicetree=%Path%\boot\tlimx8mp-evm.dtb
```

图 9

（3） 判断对应路径下，是否存在对应文件。

```
if not exist %rootfs% (
    echo rootfs not found
    goto quit
) else if not exist %modules% (
    echo modules not found
    goto quit
) else if not exist %ubootfile% (
    echo flash.bin not found
    goto quit
) else if not exist %kernel% (
    echo Image not found
    goto quit
) else if not exist %devicetree% (
    echo tlimx8mp-evm.dtb not found
    goto quit
) else (
    ::Deal with the dtb file
    start tar -zcf boot.tar.gz %Path%\boot\*
    move %rootfs% %Path%\filesystem\rootfs\rootfs.tar.gz >nul
    move %modules% %Path%\filesystem\modules\modules.tar.gz >nul

    ..\bin\uuu.exe run_ramdisk.lst
    ..\bin\uuu.exe emmc.lst
)
```

图 10

（4） 运行 uuu.exe，传入.lst 脚本。

```
    ..\bin\uuu.exe run_ramdisk.lst
    ..\bin\uuu.exe emmc.lst
)
```

图 11

3.2 run_ramdisk.lst 脚本

（1） UUU 工具可识别.lst 脚本文件。脚本文件内容首行必须为"uuu_version <version>"，"<version>"表示可解析该脚本的最低的 UUU 工具版本。

```
run_ramdisk.lst
1  uuu_version 1.5.141
2
```

图 12

（2） 脚本中首先下载并运行 flash.bin。

```
3  # download and run u-boot  mksdboot\boot\flash.bin
4  SDPS: boot -f mksdboot\boot\flash.bin
5  SDPV: delay 1000
6  SDPV: write -f mksdboot\boot\flash.bin -skipspl
7  SDPV: jump
8
```

图 13

（3） 下载内核、设备树文件、ramdisk 镜像，在 RAM 中启动 Linux 系统。

```
9   # download Kernel dtb ramdisk img and run
10  FB: ucmd setenv fastboot_buffer ${loadaddr}
11  FB: download -f Image
12  FB: ucmd setenv fastboot_buffer ${fdt_addr}
13  FB: download -f tlimx8mp-evm.dtb
14  FB: ucmd setenv fastboot_buffer ${initrd_addr}
15  FB: download -f ..\bin\fsl-image-mfgtool-initramfs-imx_mfgtools.cpio.zst.u-boot
16  FB: ucmd setenv mfgtool_args ${mfgtool_args} mfg_mmcdev=${emmc_dev}
17  FB: ucmd run mfgtool_args
18  FB: acmd booti ${loadaddr} ${initrd_addr} ${fdt_addr}
19
```

图 14

3.3 sd.lst、emmc.lst 脚本

在对应固化脚本 sd.lst 或 emmc.lst 中实现固化内核、设备树、文件系统至储存设备中。由于 sd.lst、emmc.lst 脚本文件作用类似，因此以 emmc.lst 为例进行说明。

（1） 声明支持最低的 UUU 工具版本，发送 mkpartition.sh 至 tmp 目录，修改权限后，执行脚本，传入参数为"-emmc"，执行 mkpartition.sh 脚本后，完成 eMMC 格式化和

分区。

备注：若为 sd.lst 脚本文件则传入参数为"-sd"。

```
uuu_version 1.5.141

# create partition
FBK: ucp mkpartition.sh t:/tmp
FBK: ucmd chmod 777 /tmp/mkpartition.sh
FBK: ucmd /tmp/mkpartition.sh -emmc
```

图 15

（2） 设置 mmcdev，Micro SD 卡为 1，eMMC 为 2，将 flash.bin 固化至储存设备偏移 32KByte 的地址。

```
# set mmcdev as 1
FBK: ucmd echo '1' > /tmp/mmcdev

FBK: ucp mksdboot\boot\flash.bin t:/tmp
FBK: ucmd mmc=`cat /tmp/mmcdev`; dd if=/tmp/flash.bin of=/dev/mmcblk${mmc} bs=1K seek=32
```

图 16 sd.lst

```
# set mmcdev as 2
FBK: ucmd echo '2' > /tmp/mmcdev

FBK: ucp  mksdboot\boot\flash.bin t:/tmp
FBK: ucmd mmc=`cat /tmp/mmcdev`; dd if=/tmp/flash.bin of=/dev/mmcblk${mmc} bs=1K seek=32
```

图 17 emmc.lst

（3） 将内核镜像和设备树文件固化至 mmcblk2p1 分区。

```
FBK: ucmd mmc=`cat /tmp/mmcdev`; mkfs.vfat -F 32 -n BOOT /dev/mmcblk${mmc}p1
FBK: ucmd mmc=`cat /tmp/mmcdev`; mkdir -p /mnt/fat
FBK: ucmd mmc=`cat /tmp/mmcdev`; mount -t vfat /dev/mmcblk${mmc}p1 /mnt/fat
FBK: ucp boot.tar.gz t:/tmp
FBK: ucmd tar -zxf /tmp/boot.tar.gz -C /tmp
FBK: ucmd mv /tmp/mksdboot/boot/* /mnt/fat/
FBK: ucmd umount /mnt/fat
```

图 18

（4） 将 rootfs 根文件系统固化至 mmcblk2p2 分区，将内核模块拷贝至 mmcblk2p2 分

区的"/lib/modules"目录下。

```
FBK: ucmd mmc=`cat /tmp/mmcdev`; mkfs.ext4 -F -L rootfs /dev/mmcblk${mmc}p2
FBK: ucmd mkdir -p /mnt/ext4
FBK: ucmd mmc=`cat /tmp/mmcdev`; mount /dev/mmcblk${mmc}p2 /mnt/ext4
FBK: acmd export EXTRACT_UNSAFE_SYMLINKS=1; tar -zx -C /mnt/ext4
FBK: ucp  mksdboot\filesystem\rootfs\rootfs.tar.gz t:-
FBK: Sync
FBK: ucmd sleep 1
FBK: acmd export EXTRACT_UNSAFE_SYMLINKS=1; tar -zx -C /mnt/ext4/lib/modules/
FBK: ucp  mksdboot\filesystem\modules\modules.tar.gz t:-
FBK: Sync
FBK: ucmd umount /mnt/ext4

FBK: DONE
```

图 19

3.4 mkpartition.sh 脚本

（1） 在 mkpartition.sh 脚本中实现系统分区，含 BOOT（分区 1）、roofts（分区 2）、roofts-backup（分区 3）。

```
partition_sd_card() {
    echo "Partitioning SD card..."

    # Clear the head of this SD card.
    dd if=/dev/zero of=${DEVICE} bs=1024 count=1024

    # Get the partition information
    total_size=$(($(fdisk -s ${DEVICE}) * 1024 / 512 ))

    # Offsets
    # BOOT partition: 1/5 of the total capacity of the SD card
    # rootfs partition: 2/5 of the total capacity of the SD card
    # rootfs-backup partition: 2/5 of the total capacity of the SD card
    pc1_size=$((${total_size} / 5));
    pc2_size=$((${total_size} * 2 / 5));
    pc3_size=$((${total_size} * 2 / 5));

    pc1_start=32768                         #16M
    pc2_start=$((pc1_size+pc1_start))
    pc3_start=$((pc2_size+pc2_start))

    # parted BOOT,rootfs,rootfs-backup partitions.
    PARTSTR=${pc1_start},${pc1_size},0c$'\n'${pc2_start},${pc2_size},83$'\n'${pc3_start},,83$'\n';
    echo "$PARTSTR" | sfdisk --force ${DEVICE}

    sleep 5 #FIXME Make sure partition is finished
    echo "Done!"
}
```

图 20

（2） 将三个分区进行格式化，分区 1 格式为 FAT32，分区 2 和分区 3 格式为 EXT4。

```
61  # NOTE: Not use execute(), for showing the process of mkfs.
62  format_sd_partitions() {
63      echo "-> formating BOOT partition..."
64      mkfs.vfat -F 32 -n BOOT ${DEVICE}p1
65      if [[ $? -ne 0 ]]; then
66          echo
67          err "-> ERROR: Make filesystem of boot partition failed!"
68          echo
69          exit 1
70      fi
71
72      echo "-> formating rootfs partition..."
73      mkfs.ext4 -F -L rootfs ${DEVICE}p2
74      if [[ $? -ne 0 ]]; then
75          echo
76          err "-> ERROR: Make filesystem of rootfs partition failed!"
77          echo
78          exit 1
79      fi
80
81      echo "-> formating rootfs-backup partition..."
82      mkfs.ext4 -F -L rootfs-backup ${DEVICE}p3
83      if [[ $? -ne 0 ]]; then
84          echo
85          err "-> ERROR: Make filesystem of rootfs-backup partition failed!"
86          echo
87          exit 1
88      fi
89  }
```

图 21

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com